

Binary Set

二进制集合操作

二进制集合操作

一个数的二进制表示可以看作是一个集合（表示不在集合中，表示在集合中）。比如集合，可以表示成。而对应的位运算也就可以看作是对集合进行的操作。

操作	集合表示	位运算语句
交集		$a \& b$
并集		$a \mid b$
补集	$\sim a$ （全集为二进制都是 1）	
差集		$a \& (\sim b)$
对称差		$a \wedge b$

在进一步介绍集合的子集遍历操作之前，先看位运算的有关应用例子。

模 2 的幂

一个数对 的非负整数次幂取模，等价于取二进制下一个数的后若干位，等价于和 进行与操作。



C++Python

```
1 int modPowerOfTwo(int x, int mod) { return x & (mod - 1); }
```

```
1 def modPowerOfTwo(x, mod):  
2     return x & (mod - 1)
```

于是可以知道，的非负整数次幂对它本身取模，结果为，即如果是的非负整数次幂，和的与操作结果为。

事实上，对于一个正整数，会将的最低位置零，并将后续位数全部置。因此，和的与操作等价于删掉的最低位。

借此可以判断一个数是不是的非负整数次幂。当且仅当的二进制表示只有一个 1 时，为的非负整数次幂。



C++Python

```
1 bool isPowerOfTwo(int n) { return n > 0 && (n & (n - 1)) == 0; }
```

```
1 def isPowerOfTwo(n):  
2     return n > 0 and (n & (n - 1)) == 0
```

子集遍历

遍历一个二进制数表示的集合的全部子集，等价于枚举二进制数对应掩码的所有子掩码。

掩码是一串二进制码，用于和源码进行与运算，得到屏蔽源码的若干输入位后的新操作数。

掩码对于源码可以起到遮罩的作用，掩码中的 1 意味着源码的相应位得到保留，掩码中的 0 意味着源码的相应位进

行置 操作。将掩码的若干 位改为 位可以得到掩码的子掩码，掩码本身也是自己的子掩码。

给定一个掩码，希望有效迭代 的所有子掩码，可以考虑基于位运算技巧的实现。

```
1 // 降序遍历 m 的非空子集
2 int s = m;
3 while (s > 0) {
4     // s 是 m 的一个非空子集
5     s = (s - 1) & m;
6 }
```

或者使用更紧凑的 for 语句：

```
1 // 降序遍历 m 的非空子集
2 for (int s = m; s; s = (s - 1) & m)
3 // s 是 m 的一个非空子集
```

这两段代码都不会处理等于 的子掩码，要想处理等于 的子掩码可以使用其他办法，例如：

```
1 // 降序遍历 m 的子集
2 for (int s = m;; s = (s - 1) & m) {
3     // s 是 m 的一个子集
4     if (s == 0) break;
5 }
```

接下来证明，上面的代码访问了所有 的子掩码，没有重复，并且按降序排列。

假设有一个当前位掩码，并且想继续访问下一个位掩码。在掩码 中减去，等价于删除掩码 中最右边的设置位，并将其右边的所有位变为。

为了使 变为新的子掩码，需要删除掩码 中未包含的所有额外的 位，可以使用位运算 来进行此移除。

这两步操作等价于切割掩码，以确定算术上可以取到的最大值，即按降序排列的 之后的下一个子掩码。

因此，该算法按降序生成该掩码的所有子掩码，每次迭代仅执行两个操作。

特殊情况是。在执行 之后得到，其中所有位都为。在 操作之后将得到新的 等于。因此，如果循环不以 结束，算法的循环将无法终止。

使用 表示 二进制中 的个数，用这种方法可以在 的时间复杂度内遍历集合 的子集。

遍历所有掩码的子掩码

在使用状压 DP 的问题中，有时会希望对于每个掩码，遍历掩码的所有子掩码：

```
1 for (int m = 0; m < (1 << n); ++m)
2     // 降序遍历 m 的非空子集
3     for (int s = m; s; s = (s - 1) & m)
4 // s 是 m 的一个非空子集
```

这样做可以遍历大小为 的集合的每个子集的子集。

接下来证明，该操作的时间复杂度为， 为掩码总共的位数，即集合中元素的总数。

考虑第 位，即集合中第 个元素，有三种情况：

- 在掩码 中为，因此在子掩码 中为，即元素不在大小子集中。

- 在 A 中为 1 ，但在 B 中为 0 ，即元素只在大子集中，不在小子集中。
- 在 A 和 B 中均为 1 ，即元素同时在大小子集中。

总共有 2^n 位，因此有 2^n 个不同的组合。

还有一种证明方法是：

如果掩码 A 具有 k 个 1 ，那么它有 $\binom{n-k}{k}$ 个子掩码。对于给定的 A ，对应有 $\binom{n-k}{k}$ 个掩码，那么所有掩码的总数为：

上面的和等于使用二项式定理对 $(1+x)^n$ 的展开，因此有 2^n 个不同的组合。

参考资料

本页面主要译自博文 [Перебор всех подмасок данной маски](#) 与其英文翻译版 [Submask Enumeration](#)。其中俄文版版权协议为 **Public Domain + Leave a Link**；英文版版权协议为 **CC-BY-SA 4.0**。

习题

- [Atcoder - Close Group](#)
- [Codeforces - Nuclear Fusion](#)
- [Codeforces - Sandy and Nuts](#)
- [UVa 1439 - Exclusive Access 2](#)
- [UVa 11825 - Hackers' Crackdown](#)

本页面最近更新：2024/8/12 21:39:21，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[aofall](#), [arielherself](#), [gavinliu266](#), [Great-designer](#), [jol888](#), [Menci](#), [shawlleyw](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用